

UVA 5: Funciones

cristobal.loyolam@usm.cl

21 de septiembre de 2026

- Esta semana se tomará el **control** de Funciones a aquellos estudiantes que tienen laboratorio los días miércoles-jueves-viernes. Los demás estudiantes (laboratorio lunes-martes) lo tendrán la próxima semana.
- Los **ejercicios de corrección automática** tienen plazo hasta este domingo 27 a las 23:55.
- En la ayudantía se desarrollará el trabajo preliminar de la **Tarea 2, parte 2**.
- El **Certamen 1** está fijado para el viernes 9 de octubre a las 17:30 y evaluará toda la materia, incluyendo Funciones.
- Las siguientes dos semanas estarán dedicadas a preparar el certamen.
- Recordar horarios de consultas.

Funciones: motivación

Pensemos en el funcionamiento de un auto. Es una **caja negra** que podemos entender en términos de:

- sus entradas.
- sus salidas.
- cómo se relaciona las salidas con las entradas, sin conocer el funcionamiento interno.



El funcionamiento de un auto es un ejemplo de **abstracción**:

- El conductor **no conoce los detalles** de cómo funciona internamente (no es necesario entender cómo funciona algo para saber cómo usarlo).

El funcionamiento de un auto es un ejemplo de **abstracción**:

- El conductor **no conoce los detalles** de cómo funciona internamente (no es necesario entender cómo funciona algo para saber cómo usarlo).
- El conductor sí conoce la **interfaz**.

El funcionamiento de un auto es un ejemplo de **abstracción**:

- El conductor **no conoce los detalles** de cómo funciona internamente (no es necesario entender cómo funciona algo para saber cómo usarlo).
- El conductor sí conoce la **interfaz**.
- La **relación** entre las entradas (input) y las salidas (output) es bien conocida.

Funciones: motivación

La **abstracción** facilita la **descomposición**:

- Un auto tiene cientos de componentes diferentes, fabricados por distintas compañías.



Funciones: motivación

La **abstracción** facilita la **descomposición**:

- Un auto tiene cientos de componentes diferentes, fabricados por distintas compañías.
- Cada fabricante debe saber cómo interactúa su componente con los demás.



Funciones: motivación

La **abstracción** facilita la **descomposición**:

- Un auto tiene cientos de componentes diferentes, fabricados por distintas compañías.
- Cada fabricante debe saber cómo interactúa su componente con los demás.
- Cada fabricante puede resolver sub-problemas independientes de otras partes, mientras reciba las entradas esperadas.



funciones = abstracción + descomposición

- Hay ocasiones en que queremos pensar un trozo de código como una caja negra: queremos **ocultar los detalles** tediosos y **reutilizar** esta caja negra en distintas partes de nuestro programa para evitar código duplicado.

funciones = abstracción + descomposición

- Hay ocasiones en que queremos pensar un trozo de código como una caja negra: queremos **ocultar los detalles** tediosos y **reutilizar** esta caja negra en distintas partes de nuestro programa para evitar código duplicado.
- El programador implementa los detalles y diseña la interfaz.

funciones = abstracción + descomposición

- Hay ocasiones en que queremos pensar un trozo de código como una caja negra: queremos **ocultar los detalles** tediosos y **reutilizar** esta caja negra en distintas partes de nuestro programa para evitar código duplicado.
- El programador implementa los detalles y diseña la interfaz.
- El usuario no necesita conocer ni ver los detalles, le basta con usar la caja negra, por ejemplo:

funciones = abstracción + descomposición

- Hay ocasiones en que queremos pensar un trozo de código como una caja negra: queremos **ocultar los detalles** tediosos y **reutilizar** esta caja negra en distintas partes de nuestro programa para evitar código duplicado.
- El programador implementa los detalles y diseña la interfaz.
- El usuario no necesita conocer ni ver los detalles, le basta con usar la caja negra, por ejemplo:
 - `abs(-10)`

funciones = abstracción + descomposición

- Hay ocasiones en que queremos pensar un trozo de código como una caja negra: queremos **ocultar los detalles** tediosos y **reutilizar** esta caja negra en distintas partes de nuestro programa para evitar código duplicado.
- El programador implementa los detalles y diseña la interfaz.
- El usuario no necesita conocer ni ver los detalles, le basta con usar la caja negra, por ejemplo:
 - `abs(-10)`
 - `sqrt(2)`

funciones = abstracción + descomposición

- Hay ocasiones en que queremos pensar un trozo de código como una caja negra: queremos **ocultar los detalles** tediosos y **reutilizar** esta caja negra en distintas partes de nuestro programa para evitar código duplicado.
- El programador implementa los detalles y diseña la interfaz.
- El usuario no necesita conocer ni ver los detalles, le basta con usar la caja negra, por ejemplo:
 - `abs(-10)`
 - `sqrt(2)`
 - `len('Hola mundo!')`

Usaremos la idea de **abstracción** (caja negra) para separar nuestros códigos en módulos que son:

- **autocontenidos**.
- pensados para ser **reutilizados**.

Estos módulos nos ayudarán a:

- separar un problema en partes lógicas bien definidas.
- mantener el código organizado y coherente.
- construir módulos complejos a partir de otros más simples.

- Llamaremos **función** a un trozo de código **reutilizable**.

- Llamaremos **función** a un trozo de código **reutilizable**.
- Una función encapsula un conjunto de instrucciones o cálculos que podemos aplicar sobre distintas entradas.

- Llamaremos **función** a un trozo de código **reutilizable**.
- Una función encapsula un conjunto de instrucciones o cálculos que podemos aplicar sobre distintas entradas.
- Al **definir** una función le estamos diciendo a Python que almacene cierto código en memoria.

- Llamaremos **función** a un trozo de código **reutilizable**.
- Una función encapsula un conjunto de instrucciones o cálculos que podemos aplicar sobre distintas entradas.
- Al **definir** una función le estamos diciendo a Python que almacene cierto código en memoria.
- Las funciones sólo son útiles cuando las **llamamos** o invocamos.

- Llamaremos **función** a un trozo de código **reutilizable**.
- Una función encapsula un conjunto de instrucciones o cálculos que podemos aplicar sobre distintas entradas.
- Al **definir** una función le estamos diciendo a Python que almacene cierto código en memoria.
- Las funciones sólo son útiles cuando las **llamamos** o invocamos.
- Una función se define sólo 1 vez pero puede ser llamada muchas veces.

Funciones: definición

La definición de una función contiene:

- un **nombre** que nos permitirá invocarla posteriormente.

nombre



```
def es_par(n):  
    if n % 2 == 0:  
        return True  
    else:  
        return False
```

Funciones: definición

La definición de una función contiene:

- un **nombre** que nos permitirá invocarla posteriormente.
- un conjunto de **parámetros** (0, 1 o más) que corresponden a los datos de entrada (pueden ser de cualquier tipo).

nombre **parámetro(s)**



```
def es_par(n):  
    if n % 2 == 0:  
        return True  
    else:  
        return False
```

Funciones: definición

La definición de una función contiene:

- un **nombre** que nos permitirá invocarla posteriormente.
- un conjunto de **parámetros** (0, 1 o más) que corresponden a los datos de entrada (pueden ser de cualquier tipo).
- un **cuerpo** que corresponde al conjunto de instrucciones a ejecutar cuando hacemos un llamado a la función.

```
      nombre parámetro(s)
      └──┬──┘ └──┬──┘
def es_par(n):
    if n % 2 == 0:
        return True
    else:
        return False
```

} cuerpo de la función (indentado)

Funciones: definición

La definición de una función contiene:

- un **nombre** que nos permitirá invocarla posteriormente.
- un conjunto de **parámetros** (0, 1 o más) que corresponden a los datos de entrada (pueden ser de cualquier tipo).
- un **cuerpo** que corresponde al conjunto de instrucciones a ejecutar cuando hacemos un llamado a la función.
- un valor de **retorno** que puede ser de cualquier tipo: int, float, bool o str.

```
      nombre parámetro(s)
      {      }
def es_par(n):
    if n % 2 == 0:
        return True
    else:
        return False
    }
    {
```

cuerpo de la
función
(indentado)

valor de retorno

Funciones: llamado

- Ya vimos cómo definir funciones, pero ¿cómo podemos usarlas?
- El código de una función se ejecutará sólo al momento de **llamar** o invocar a dicha función.
- Para esto, sólo debemos escribir el nombre de la función y pasarle valores como parámetros entre paréntesis, por ejemplo:

es_par (17)
nombre valor
 pasado
 como
 parámetro

¿Qué hará el siguiente programa?

```
1  def es_par(n):  
2      if n % 2 == 0:  
3          return True  
4      else:  
5          return False  
6  
7  es_par(17)  
8  es_par(5)
```

¿Qué hará el siguiente programa?

```
1  def es_par(n):  
2      if n % 2 == 0:  
3          return True  
4      else:  
5          return False  
6  
7  print(es_par(17))  
8  print(es_par(5))
```

Funciones: ¿cómo escribirlas?

Al momento de definir una función debemos tener en cuenta lo siguiente:

- ¿Cuál es el problema que queremos resolver?

Funciones: ¿cómo escribirlas?

Al momento de definir una función debemos tener en cuenta lo siguiente:

- ¿Cuál es el problema que queremos resolver?
- ¿Qué parámetro(s) recibe la función y de qué tipo es/son?

Funciones: ¿cómo escribirlas?

Al momento de definir una función debemos tener en cuenta lo siguiente:

- ¿Cuál es el problema que queremos resolver?
- ¿Qué parámetro(s) recibe la función y de qué tipo es/son?
- ¿Cómo resolveremos el problema?

Funciones: ¿cómo escribirlas?

Al momento de definir una función debemos tener en cuenta lo siguiente:

- ¿Cuál es el problema que queremos resolver?
- ¿Qué parámetro(s) recibe la función y de qué tipo es/son?
- ¿Cómo resolveremos el problema?
- ¿Qué valor de salida debe producir mi función y de qué tipo es?

Funciones: ¿cómo escribirlas?

Al momento de definir una función debemos tener en cuenta lo siguiente:

- ¿Cuál es el problema que queremos resolver?
- ¿Qué parámetro(s) recibe la función y de qué tipo es/son?
- ¿Cómo resolveremos el problema?
- ¿Qué valor de salida debe producir mi función y de qué tipo es?

Ejercicio:

- 1 Queremos escribir una función que calcule la suma de todos los números impares entre los límites a y b (ambos incluidos).
- 2 Luego, queremos que nuestro programa imprima la suma de todos los impares entre 2 y 7.

Paso de parámetros y valor de retorno

Considere el siguiente código:

```
1  def sumar(a, b):  
2      suma = a + b  
3      return suma  
4  num1 = 2  
5  num2 = 3  
6  s = sumar(num1, num2)  
7  print(s)
```

- ¿Qué valor tiene la variable `suma` después de que termina la función? ¿Por qué?
- ¿Qué imprime el programa en la pantalla? ¿Por qué?

print vs return

¿Qué imprimen en la pantalla los siguientes programas?

```
1 def mi_funcion(x):  
2     y = 2*x + 3  
3     return y  
4 a = 5  
5 print(a, mi_funcion(a))
```

```
1 def mi_funcion(x):  
2     y = 2*x + 3  
3     print(y)  
4 a = 5  
5 print(a, mi_funcion(a))
```

print vs return

¿Qué imprimen en la pantalla los siguientes programas?

```
1 def mi_funcion(x):  
2     y = 2*x + 3  
3     return y  
4 a = 5  
5 print(a, mi_funcion(a))
```

5 13

```
1 def mi_funcion(x):  
2     y = 2*x + 3  
3     print(y)  
4 a = 5  
5 print(a, mi_funcion(a))
```

13
5 None

¿Qué errores encuentra en la siguiente función?

```
1      # Retorna el MCD de a y b mediante el algoritmo de Euclides
2      # El algoritmo en sí (el ciclo) no es importante
3      # para el ejercicio
4      def euclides(a, b):
5          a = int(input('a: '))
6          b = int(input('b: '))
7          while b != 0:
8              resto = a % b
9              a = b
10             b = resto
11         print(a)
```

Solución:

```
1  def euclides(a, b):  
2      # Sin input, ya que los valores vienen  
3      # como parámetros de la función  
4      while b != 0:  
5          resto = a % b  
6          a = b  
7          b = resto  
8      return a # return en vez de print
```

Ejemplo: cálculo de distancia

El siguiente programa utiliza una función para calcular la distancia entre dos puntos cuyas coordenadas son ingresadas como entrada. Completélo para que funcione correctamente:

```
1  from math import sqrt
2  def distancia(x1, y1, x2, y2):
3      t1 = (____)**2
4      t2 = (____)**2
5      d = sqrt(t1 + t2)
6      -----
7
8  a = float(input('Coordenada x del primer punto: '))
9  b = float(input('Coordenada y del primer punto: '))
10 c = float(input('Coordenada x del segundo punto: '))
11 d = float(input('Coordenada y del segundo punto: '))
12 ____ = distancia(____)
13 print('La distancia es:', dist)
```

Ejemplo: cálculo de distancia

Solución:

```
1  from math import sqrt
2  def distancia(x1, y1, x2, y2):
3      t1 = (x1 - x2)**2
4      t2 = (y1 - y2)**2
5      d = sqrt(t1 + t2)
6      return d
7
8  a = float(input('Coordenada x del primer punto: '))
9  b = float(input('Coordenada y del primer punto: '))
10 c = float(input('Coordenada x del segundo punto: '))
11 d = float(input('Coordenada y del segundo punto: '))
12 dist = distancia(a, b, c, d)
13 print('La distancia es:', dist)
```

Orden de los parámetros

Complete el siguiente programa para calcular la pendiente de una recta que pasa por los puntos (0, 0) y (1, 1):

```
1  def pendiente(x1, x2, y1, y2):  
2      m = (y2 - y1) / (x2 - x1)  
3      return m  
4  ----- pendiente(-----)  
5  print(p)
```

Orden de los parámetros

Solución:

```
1  def pendiente(x1, x2, y1, y2):  
2      m = (y2 - y1) / (x2 - x1)  
3      return m  
4  p = pendiente(0, 1, 0, 1)  
5  print(p)
```

Alcance de variables y paso de parámetros

¿Qué imprime el siguiente programa?

```
1  def es_divisible(a, b):
2      if a % b == 0:
3          return True
4      else:
5          return False
6
7  a = 5
8  b = 10
9  if es_divisible(b, a):
10     print(b, 'es divisible por', a)
11 else:
12     print(b, 'no es divisible por', a)
```

Alcance de variables y paso de parámetros

El siguiente es otro intento por implementar una función que determine si un número es divisible por otro o no. A diferencia de la versión anterior, se omitió el `else`:

```
1  def es_divisible(a, b):  
2      if a % b == 0:  
3          return True  
4      return False
```

¿Está correcta la función o tiene errores? ¿Es necesario el `else`?
¿Por qué?

Complete el ruteo del programa que calcula, usando funciones, el coeficiente binomial definido como:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

```
def factorial(n):  
    f = 1  
    i = 2  
    while i <= n:  
        f *= i  
        i += 1  
    return f  
  
def coeficiente_binomial(n, k):  
    a = factorial(n)  
    b = factorial(k)  
    c = factorial(n - k)  
    return a / (b * c)  
  
coef = coeficiente_binomial(3, 2)  
print(coef)
```

